DAY 13 | POST-READ

Prompt Engineering: Recap and Beyond

What we covered today, plus ideas to stretch your understanding.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

Part A: Summary of today

Topic	What to remember
Prompt anatomy	Role, task, context, format, examples, constraints.
Guidelines	Be specific, give context, show format, split tasks, say what to do when unsure, iterate.
Zero, one, few shot	Examples teach format and edge cases, at the cost of input tokens.
CoT and ReAct	Steps before answers; thought, action, observation with tools and a step limit.
Self-consistency and reflection	Majority of several answers; draft, critique, revise.
Reasoning models (NEW v4)	Give goal, constraints and success criteria; choose reasoning effort.
Structured outputs (NEW v4)	JSON Schema in the request, Pydantic validation in your code.
Templates and caching (NEW v4)	Versioned templates; stable prefix first so it can be cached.
Prompt injection (NEW v4)	Delimit untrusted text, treat it as data, validate the output.

Part B: Going beyond today

1. System and user messages

Chat APIs take a list of messages. Put stable rules in the system message and task details in the user message. That split also helps caching.

2. Prompts are code

Keep prompts in version control, test them against a fixed set of examples, and review changes like code. You will build prompt regression tests and quality gates on Day 32.

3. When structured output fails

Even with a schema, handle refusals, empty answers and timeouts. Retry once, then fall back to a safe default such as category "other" with a flag for a human.

4. Indirect injection

Injection can also arrive through documents, web pages or tool results the model reads, not only through the user. This matters for RAG on Day 18 and agents on Days 26 to 28. Day 30 covers defences in depth.

5. Cost of techniques

Few-shot adds input tokens. Self-consistency multiplies calls. Reasoning effort adds output tokens. Measure quality and cost together, then pick the cheapest prompt that meets the bar.

Part C: Check yourself

- When would you choose few-shot over zero-shot?
- What are the three steps that repeat in ReAct?
- How should you prompt a reasoning model differently from a standard model?
- Why validate structured output with Pydantic if the API already follows the schema?
- Where should the stable part of a prompt go, and why?
- Give one sign that a ticket contains a prompt injection attempt.

Part D: Practice ideas for tonight

- Rewrite one prompt three ways (zero-shot, few-shot, structured output) and compare outputs and tokens.
- Add an injection test ticket to your SmartDesk classifier and check it is labelled safely.
- Turn your best prompt into a versioned template with three test inputs.

Coming up on Day 15: testing with PyTest and an introduction to data engineering.

Otsukaresama deshita (お疲れ様でした): thank you for your hard work today